Complex Networks and Intrusion Detection Systems with AI

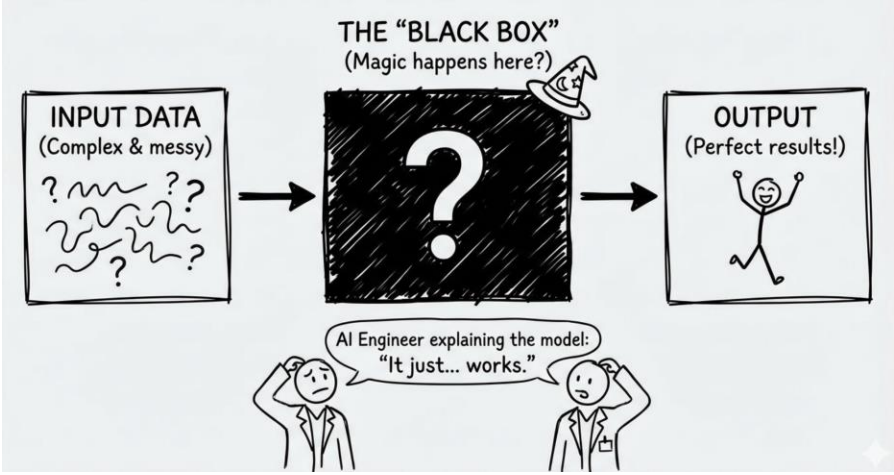
WOOD, JARED

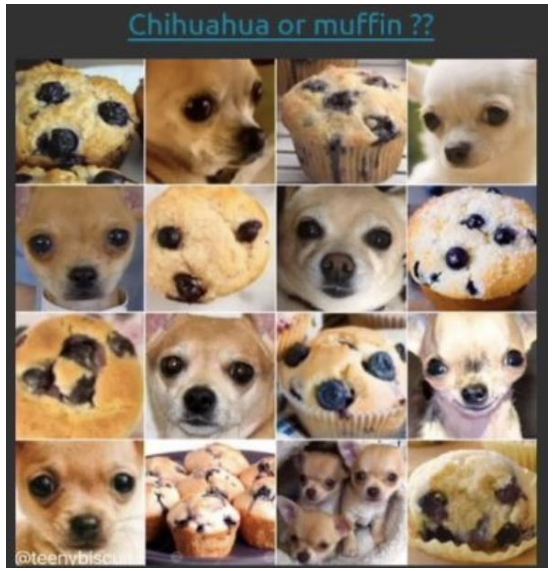
The Problem (Motivation)

- INTERPRETABILITY PARADOX: Better models usually become more ambiguous

- VULNERABILITY TO POISONING: Reliance on "rate-of-change" (Stats, probability, calculus) leaves models susceptible

- GOAL: Implement a structural ground truth that is explainable for better performance

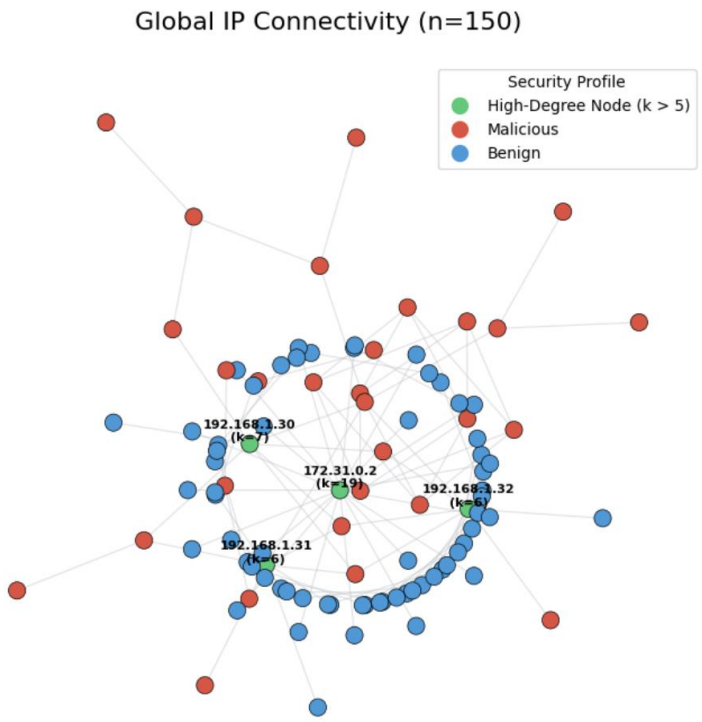




The Solution (Topological Context)

- **FEATURE ENGINEERING**: Implement topological definitions via Complex Networks
- **INDEPENDENT VAR**: Complex Network vs. Tabular dataset based on same origin dataset
- **DEPENDENT VAR**: Bare-bones identical ML and NN models
- **HYPOTHESIS**: Stealthy attacks (Recon/Theft) will follow more topology allowing for easier detection

Graph Status: Not Connected



Key Takeaway: The graph is not connected, thus we will need to try and group all the connected graphs in order to properly apply complex network ideology

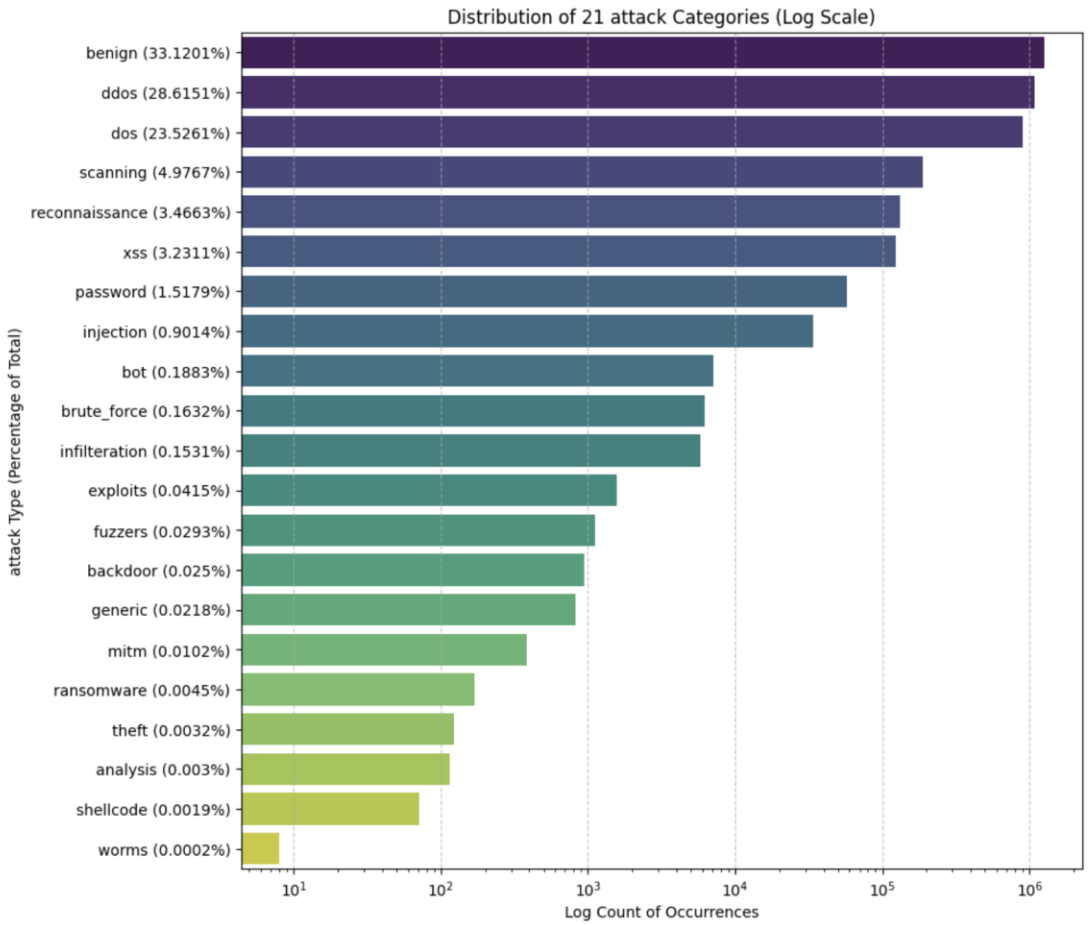
ipv4_src_addr	ipv4_dst_addr	in_bytes	out_bytes	attack
192.168.1.31	192.168.35.45	44	0	scanning
192.168.1.34	192.168.1.49	44	0	benign
192.168.100.149	192.168.100.5	44	40	reconnaissance
172.31.69.13	172.31.69.7	44	40	infilteration
192.168.100.3	192.168.100.148	44	180	ddos
192.168.1.31	192.168.1.195	44	44	password
192.168.100.7	192.168.100.148	44	180	dos

The Dataset & The Gap

- **Kaggle:** NF-UQ-NIDS-v2 Network Intrusion Dataset by Arya Shah (75,987,976 x 46 entries)

- **IMBALANCED DATASET:** 1/3 of the data is benign, 2/3 is malicious

- **NEEDLE-IN-A-HAYSTACK:** 14/20 malicious types take up ~1.5% of the dataset

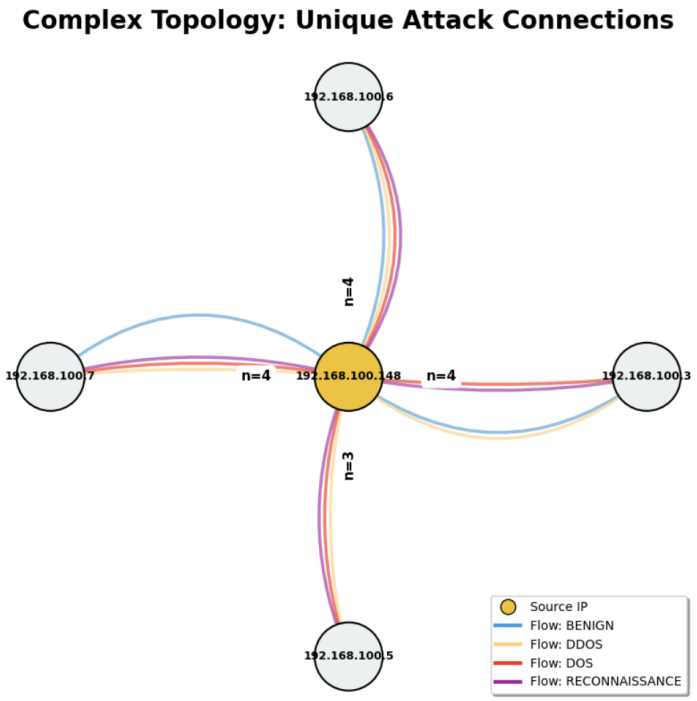


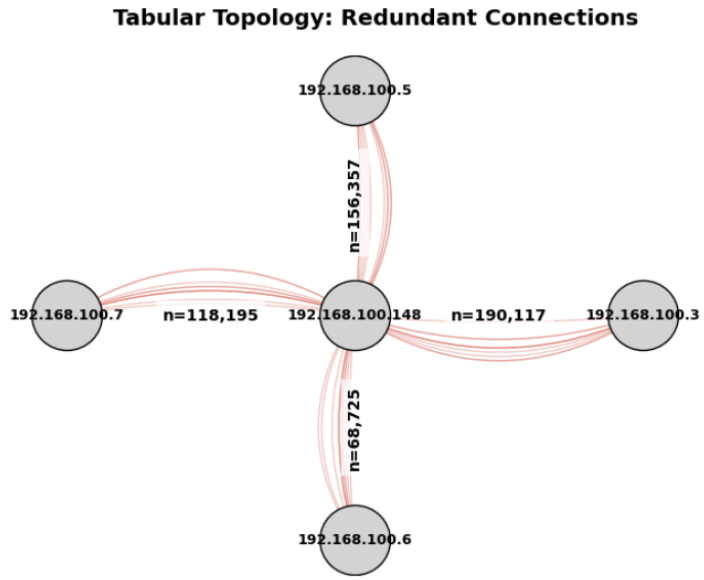
From Packet to Topology

- MATRIX INCOMPATIBILITY: Instances of thousands of edges between two nodes

- QUASI-FIX: Grouping by attack type and aggregating weighted edges to capture interaction

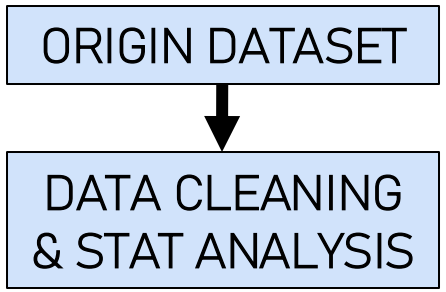
- GRAPH TYPE: Directed and weighted graph





The Features

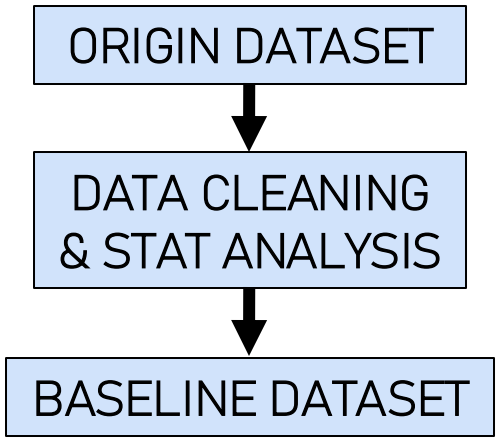
- Acquire the original NF-UQ-NIDS-v2
- 75,987,976 x 46 entries



The Features

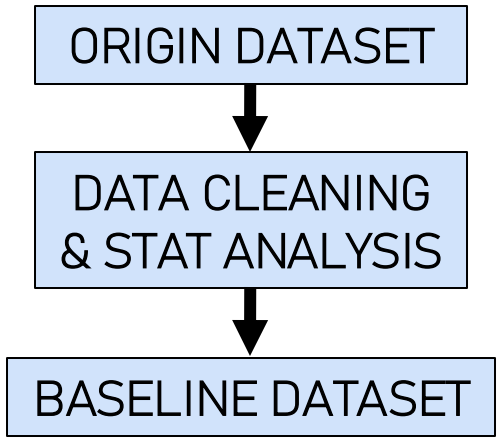
- Reduce to 5% of data
- Remove statistically insignificant and non-associative features

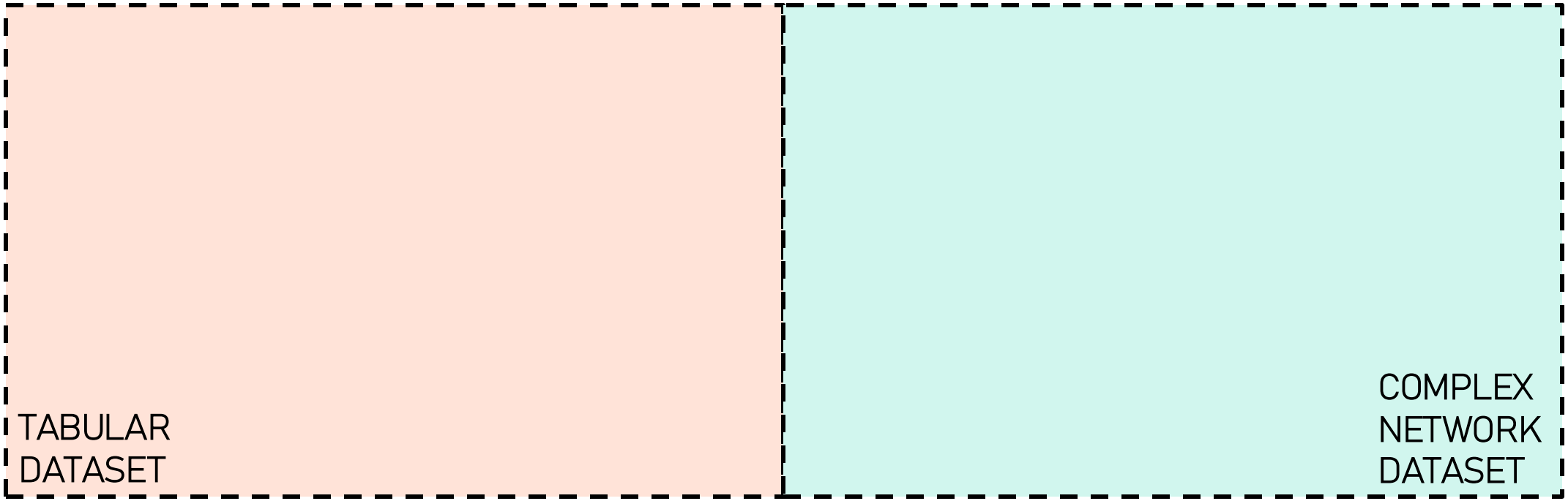
The Features



- Establish baseline dataset

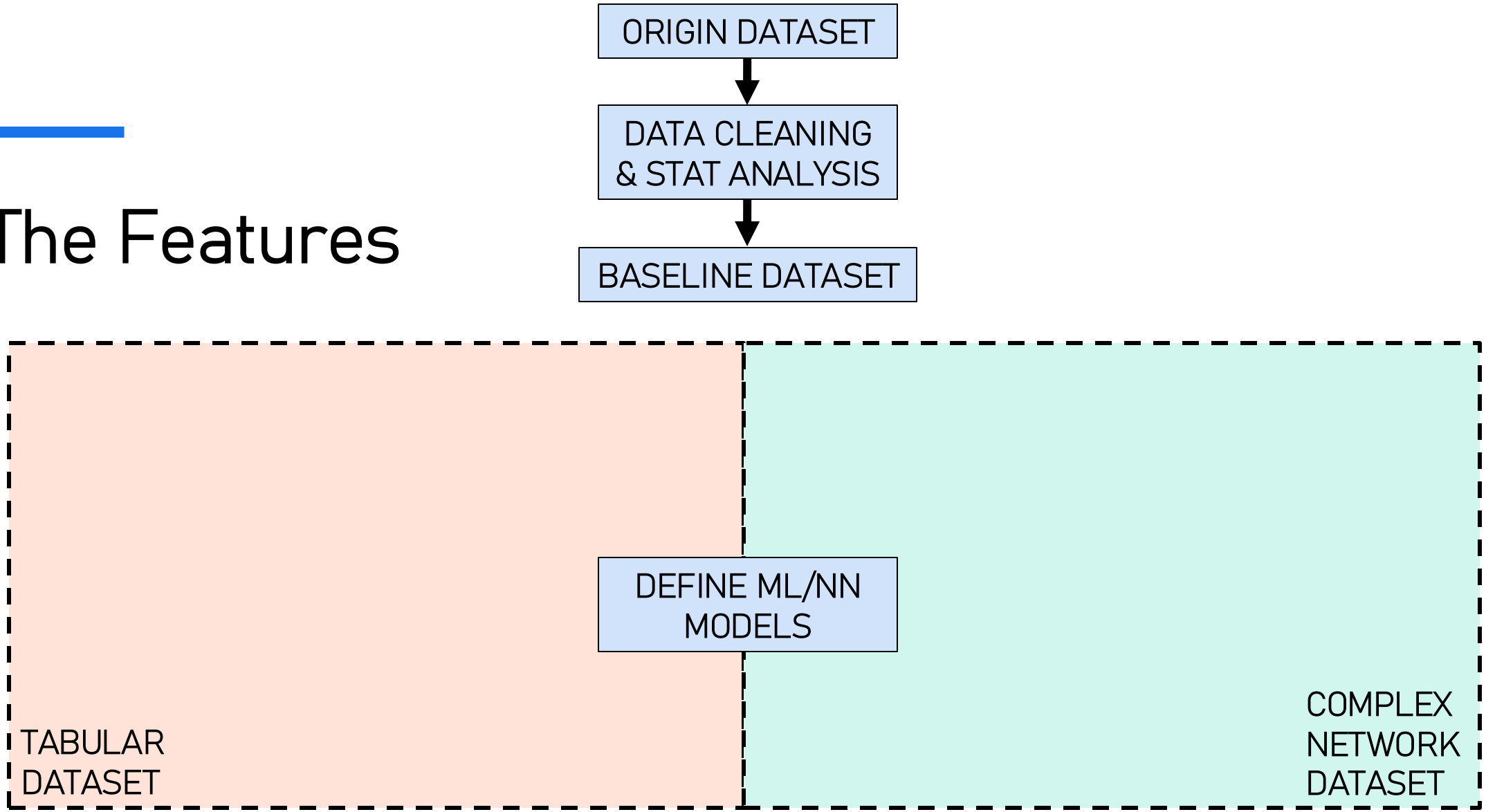
The Features





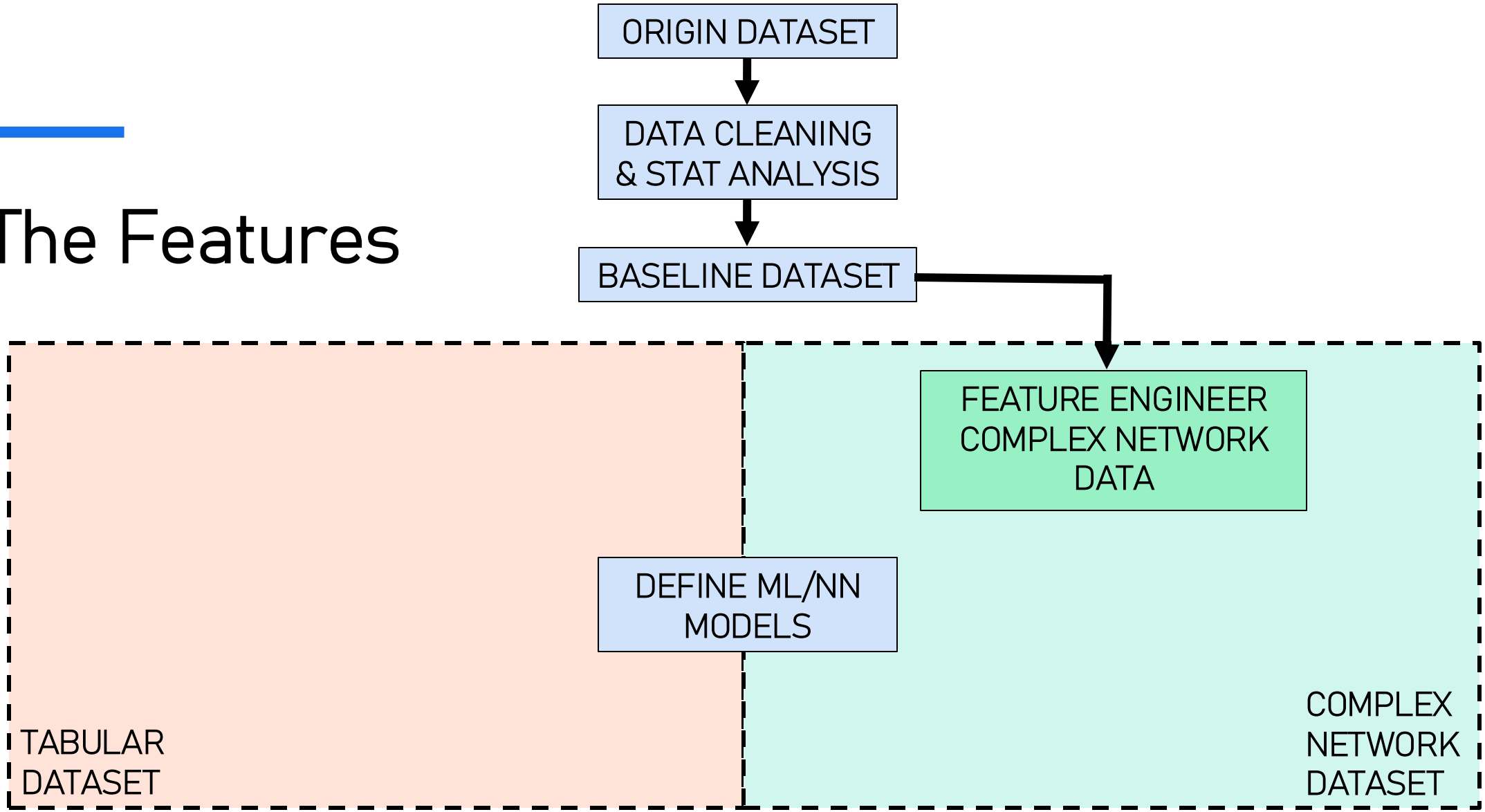
- Separate process for non-complex network dataset and complex dataset

The Features



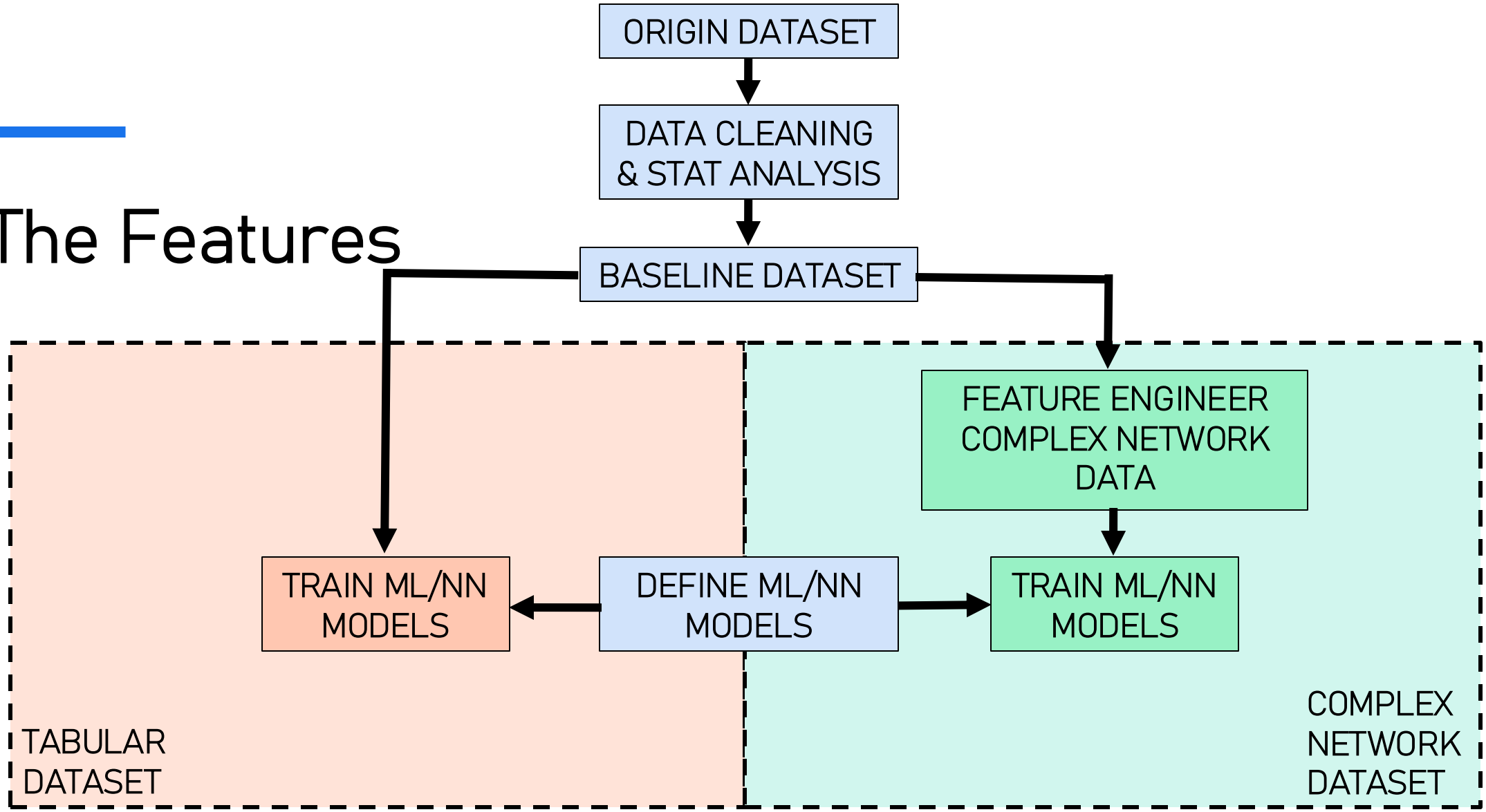
- Establish "bare-bones" ML and NN models

The Features



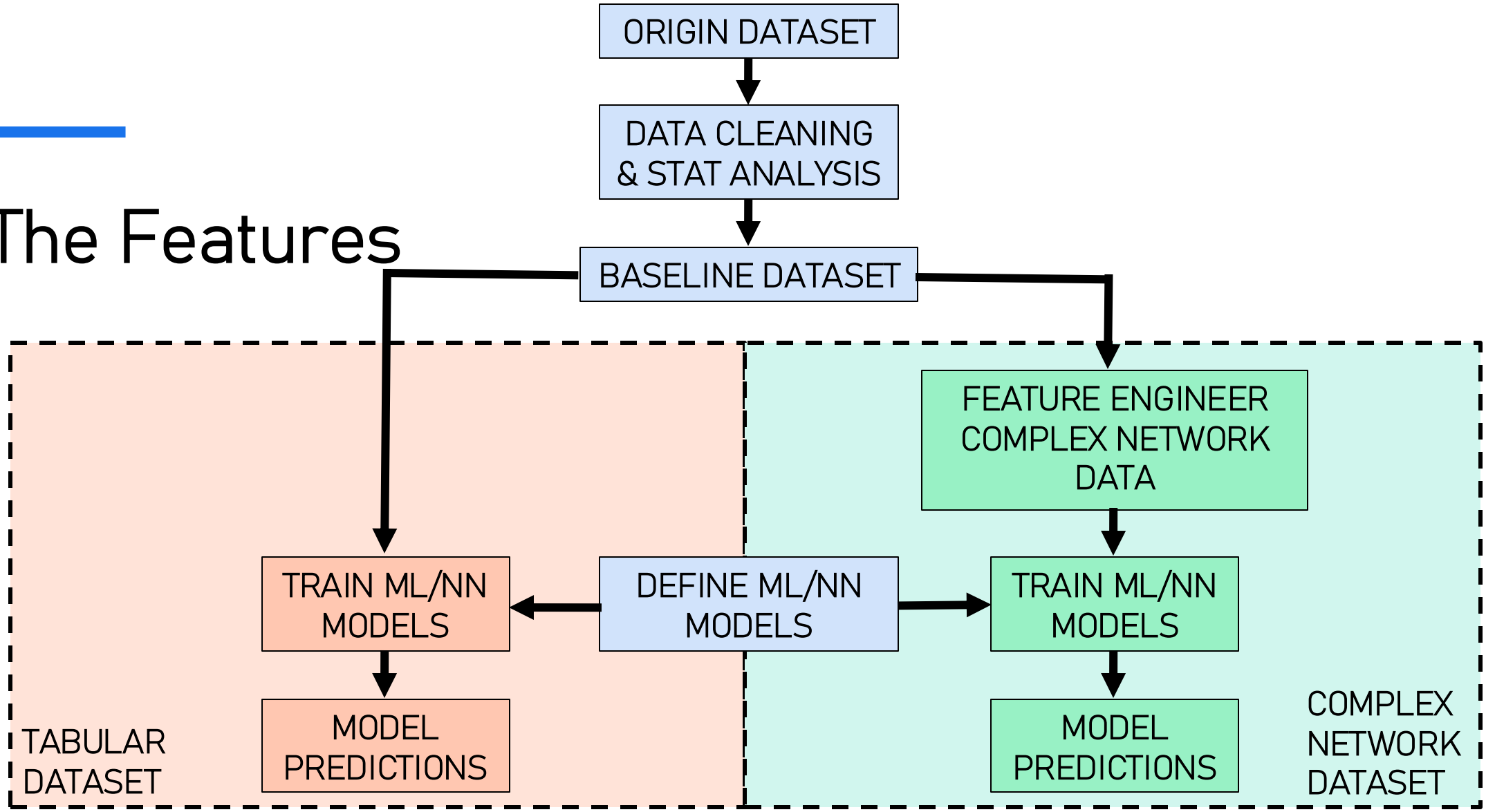
- Create Complex Network Dataset
- iGraph, numpy, scipy provide most application functions

The Features



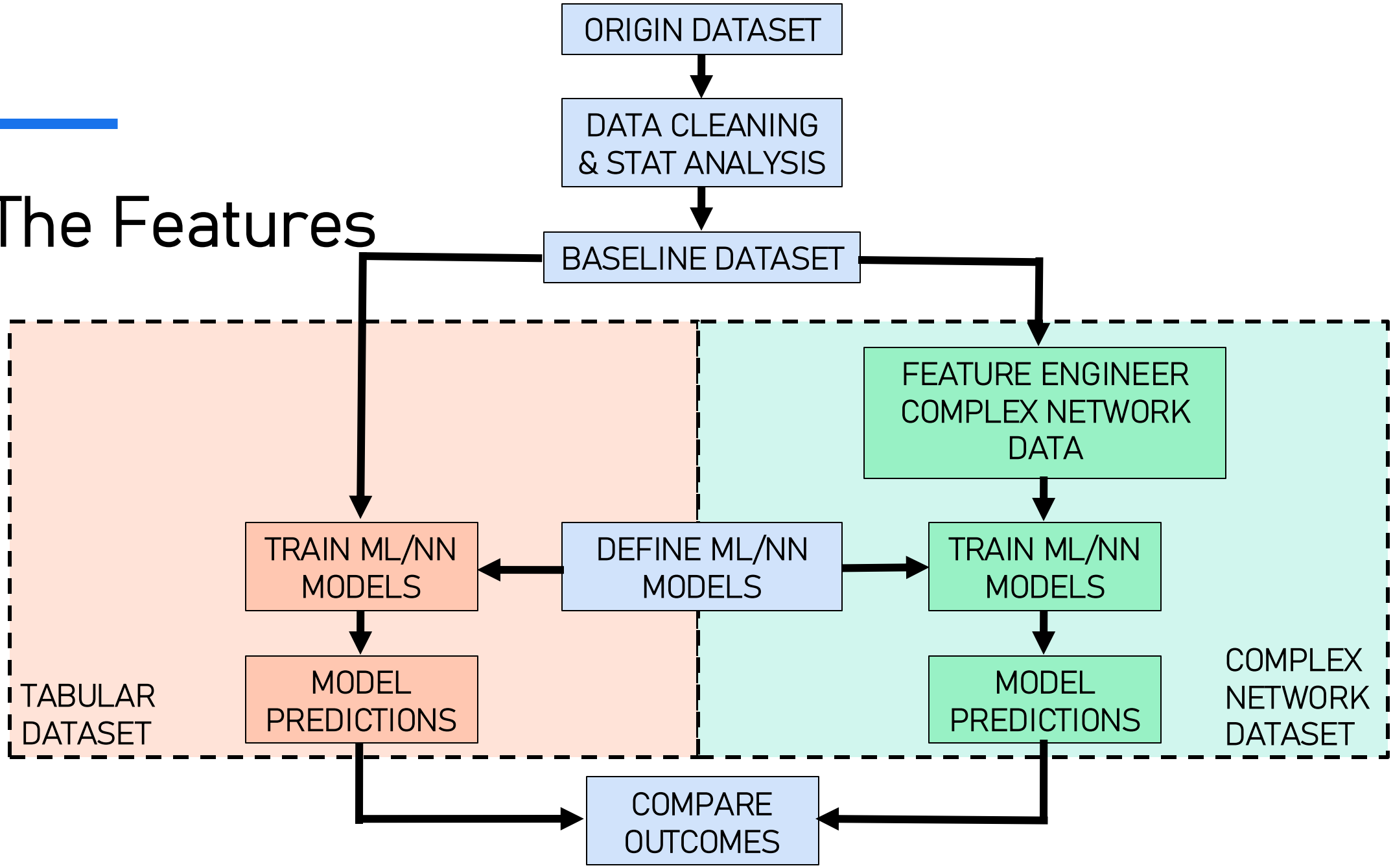
- Train ML/NN models on respective datasets

The Features



- Make respective predictions of attack-type

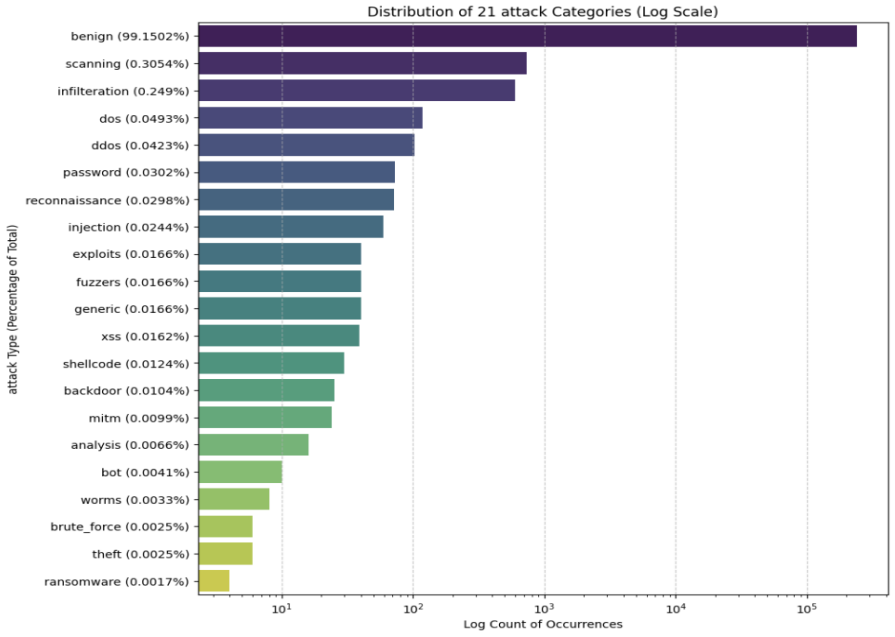
The Features

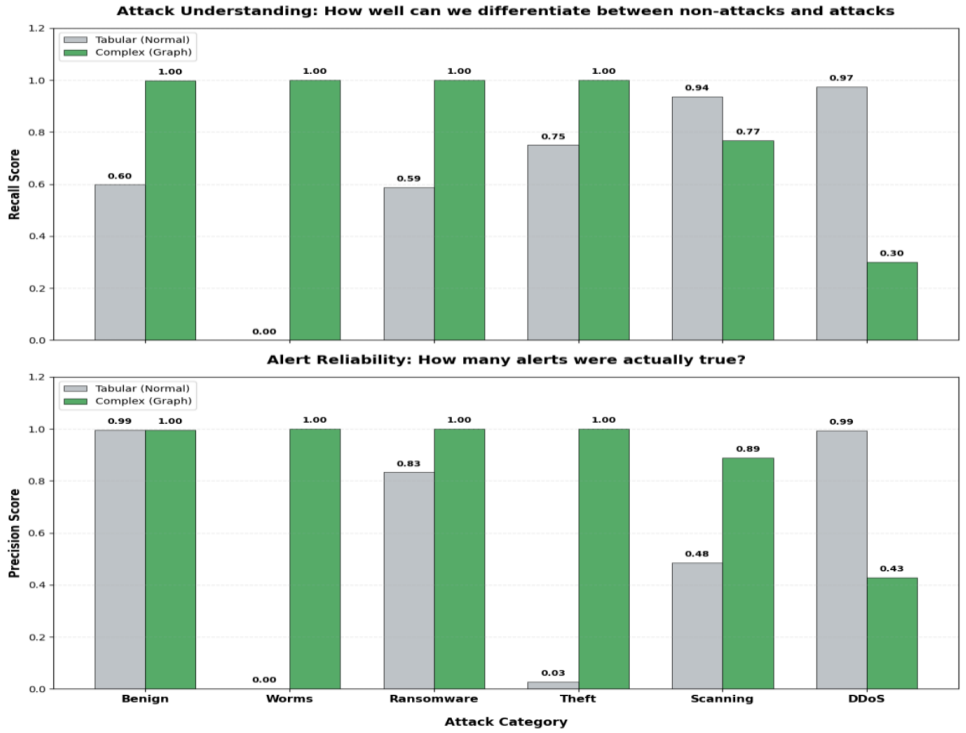


- Compare prediction efficacy

The "Smoking Gun"

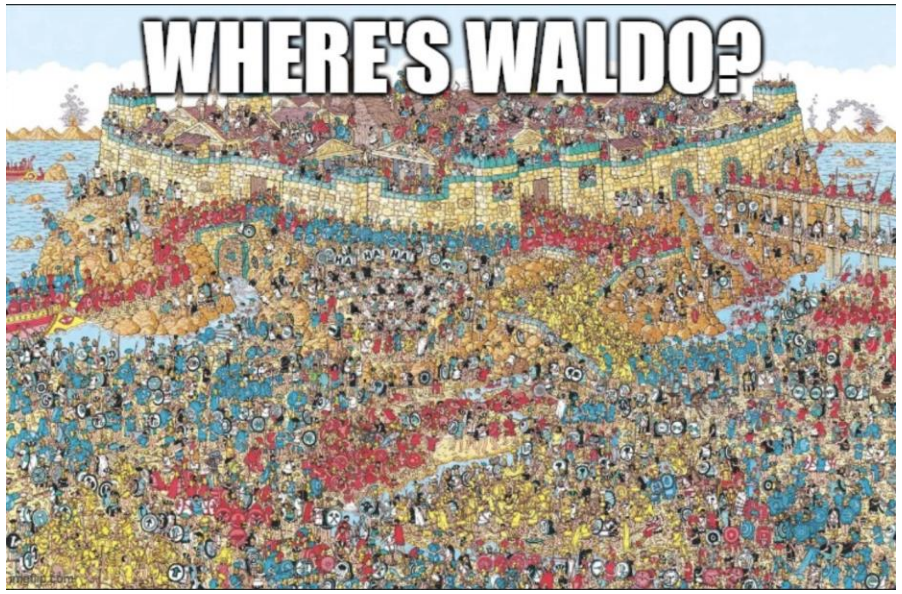
- **DATASET IMBALANCE:** Complex Network dataset nearly 99% benign, but still showed significant performance increase
- **UNSEEN PATTERNS:** Tabular-based models had significant difficulty understanding "stealthy" attacks
- **TOPOLOGICAL PATTERNS:** Complex Network-based models nearly perfectly identified most "stealthy" attacks

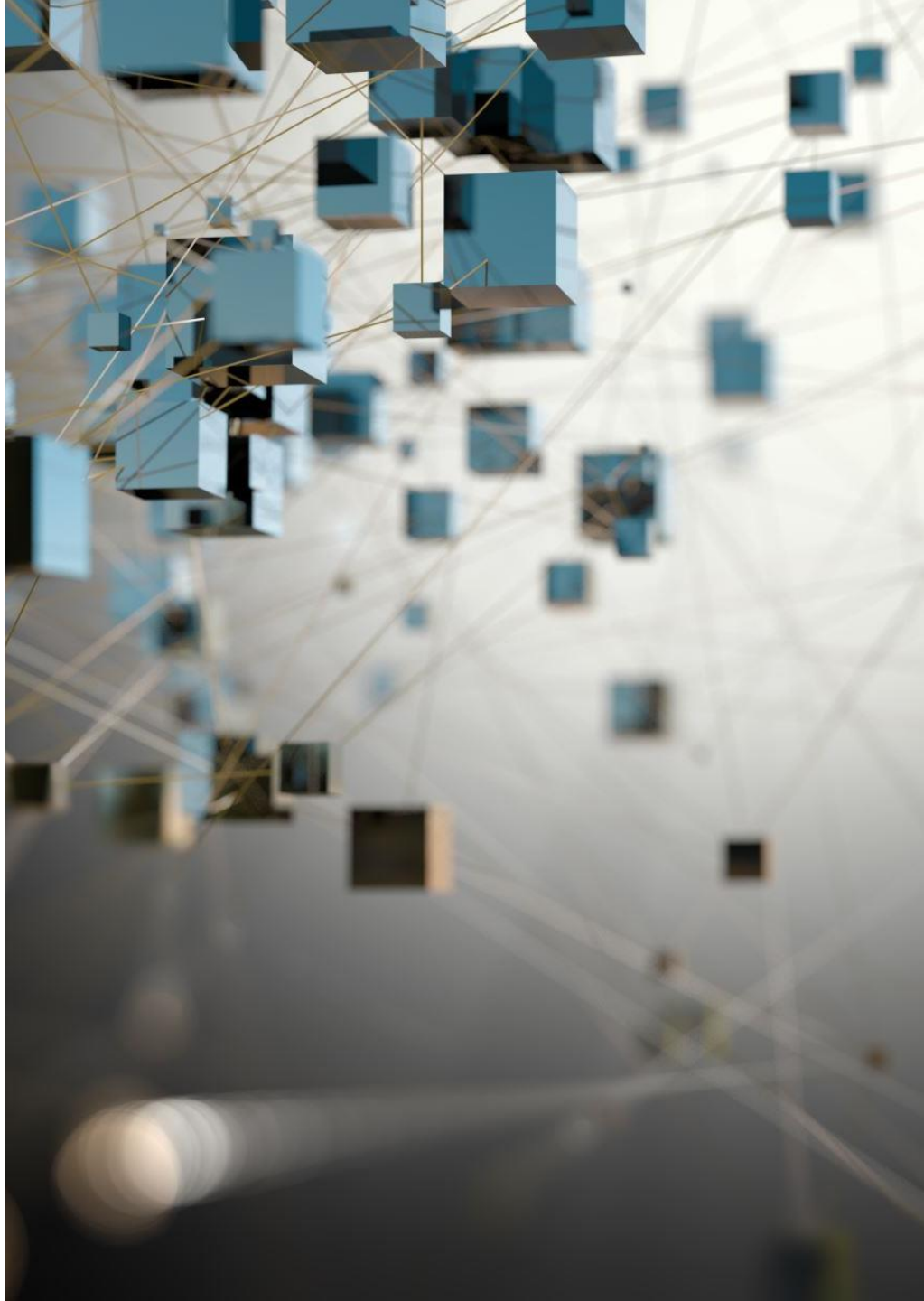




The Honest Trade-Off

- POTENTIAL OVERFITTING: Overly analytical for "stealthy" attacks may cause loss in detection of "obvious" attacks
- TIME/RESOURCE REQ: Naturally, implementing complex network information takes a scaling amount of time and computational resources before ready for use
- OVERLY GENERAL APPLICATION: I did not have the time to observe things individually nor verify applications/functions for legitimacy nor have SME in cybersecurity



Key Takeaways

- COMPLEX NETWORK SIGNIFICANCE: Can better identify hidden patterns for better understanding and ingestion of large data
- FURTHER RESEARCH NECESSARY: Much of the project was done in general and quick decisions due to time limitations
- POTENTIAL APPLICATIONS: Medical physics (Cancer/Tumors/Illness), traffic congestion/analysis, better corroboration of intelligence reports and anomalies, more efficient allocation of resources/learning (Neuromorphic computing)